Dresden University of Technology

Computational Logic ∴ Group

Sarah Gaggl
Faculty of Computer Science, Institute of Artificial Intelligence, Computational Logic Group

Tabu Search

Lecture 4 // Advanced Problem Solving and Search, WS 2025/26

Agenda

1. Introduction
2. Uninformed Search versus Informed Search (Best First Search, A* Search, Dynamic Programming)
3. Local Search, Stochastic Hill Climbing, Simulated Annealing
4. Tabu Search
5. Evolutionary Algorithms/ Genetic Algorithms
6. Answer-set Programming (ASP)
7. Constraint Satisfaction (CSP)
8. Structural Decomposition Techniques (Tree/Hypertree Decompositions)

Previously...

Local search and hill-climbing:

- Iteratively move to a better neighbour of the current solution.
- Terminate when no improvement is found — stuck at a **local optimum**.
- Restarting from multiple random points helps, but gives no guarantee of finding the global optimum.

Stochastic hill-climber:

- Sample **one random neighbour** per step.
- Accept it with probability $p = \frac{1}{1+e^{(eval(v_C)-eval(v_N))/T}}$.
- Can accept **worse moves** — escape from local optima is possible. Temperature T stays **fixed**.

Simulated Annealing (SA):

- Always accepts better solution and worse with probability (like stochastic hill-climber), but temperature T **decreases** over time (cooling schedule).
- High T: exploration — almost any move accepted.
- Low T: exploitation — only improving moves accepted.
- **Stochastic** and memoryless.

The key limitation

Both hill-climbing and SA are **memoryless**: they make no use of the history of the search. Today: **Tabu Search** — escaping local optima through **memory**.

Tabu Search

Main Idea

- A memory forces the search to explore new areas of the search space
- Memorize solutions that have been examined recently. They become tabu points in next steps
- Tabu search is deterministic

Tabu Search (High-Level)

Algorithm tabu search

```
init x; best ← x; T ← ∅
repeat
   N*(x) ← {y ∈ N(x) |
        y ∉ T or aspiration(y)}
   x ← arg max_{y∈N*(x)} eval(y)
   if eval(x) > eval(best) then
      best ← x
   end if
   update T (add move; expire old)
until halting criterion
return best
```

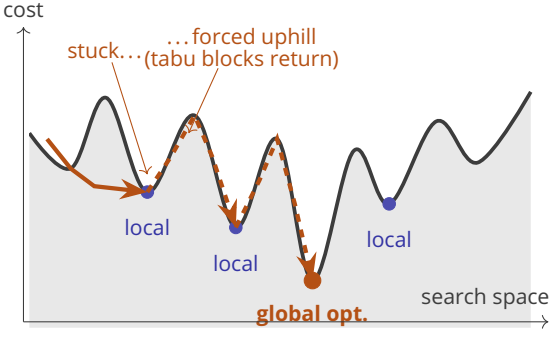
Three key components:

- **Admissible neighbourhood** $N^*(x)$: neighbours that are either **not tabu** or satisfy the aspiration criterion.
- **Best-admissible move**: always move to the best non-tabu neighbour — even if it is worse than the current solution.
- **Aspiration criterion**: override tabu status when a move leads to a solution better than the best found so far.

Note

Tabu search is **deterministic**: the same start and tabu list always produce the same sequence of moves.

Tabu Search: Energy Landscape



What the tabu list does:

- At a **local optimum** no improving move exists — but tabu search must still move.
- It picks the **best non-tabu** neighbour, even if worse than current.
- The tabu list **blocks the return**, preventing immediate cycling back.
- This forces the search **uphill and onward** into new regions.

Contrast with SA:

- SA escapes **probabilistically** via temperature.
- Tabu search escapes **deterministically** via memory.
- Neither guarantees the global optimum.

Tabu Search and SAT

- SAT problem with $n = 8$ variables
- Initial (random) assignment $\mathbf{x} = (0, 1, 1, 1, 0, 0, 0, 1)$
- Evaluation function: weighted sum of number of satisfied clauses. Weights depend on the number of variables in the clause
- Maximize evaluation function (i.e. we're trying to satisfy all clauses)
- Random assignment provides $eval(\mathbf{x}) = 27$
- Neighborhood of $\mathbf{x}$ consists of 8 solutions. Evaluate them and select best
- **At this stage, it is the same as hill-climbing**
- Suppose flipping 3rd variable generates best evaluation ($eval(\mathbf{x}') = 31$)
- Memory keeps track of actions

Recency-based Memory

- Index of flipped variable + time when it was flipped
- **Differentiate between older and more recent flips**
- SAT: time stamp for each position of solution vector M (initialized to 0)
- Value of time stamp provides information on recency of flip at position

Memory Vector

$M(i) = j$ (when $j \neq 0$)
j is most recent iteration when i-th bit was flipped

Recency-based Memory

- Index of flipped variable + time when it was flipped
- **Differentiate between older and more recent flips**
- SAT: time stamp for each position of solution vector M (initialized to 0)
- Value of time stamp provides information on recency of flip at position

Memory Vector

$M(i) = j$ (when $j \neq 0$)
j is most recent iteration when i-th bit was flipped

Assume information is stored for at most 5 iterations.

Alternative Interpretation

$M(i) = j$ (when $j \neq 0$)
i-th bit was flipped $5 - j$ iterations ago

Different Interpretations

Example

0	0	1	0	0	0	0	0

1st Variant

- Stores iteration number of most recent flip
- Requires a current iteration counter t which is compared with memory values
- If $t - M(i) > 5$ forget
- Only requires updating a single entry, and increase the counter
- **Used in most implementations**

Different Interpretations

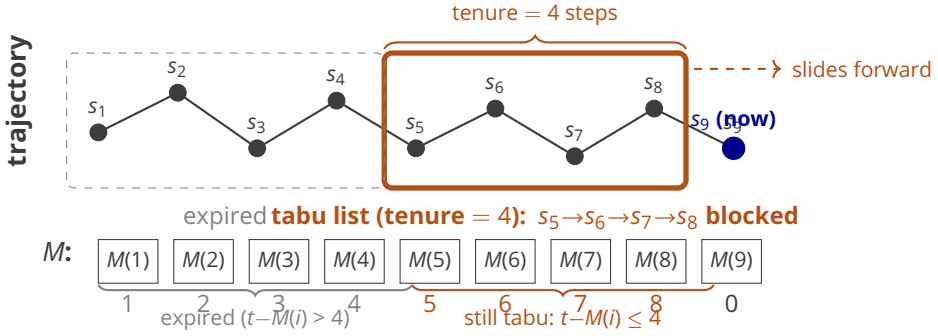
Example

0	0	5	0	0	0	0	0

2nd Variant

- Values are interpreted as number of iterations for which a position is not available
- **All** nonzero entries are decreased by one at every iteration

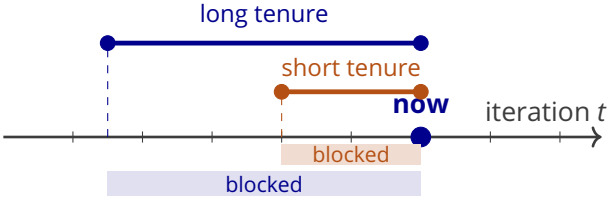
The Tabu List as a Sliding Window



Key insight

At each step the window **slides forward** by one: the oldest move expires and the newest enters. The **tenure** is the window width — it controls how many recent moves are simultaneously blocked, preventing cycling while freeing older moves for reuse.

Tabu Tenure — How Long Should a Move Stay Tabu?



The **tabu tenure** is the number of iterations a move remains forbidden after it is made.

	Short	**Long**
Cycling risk	Higher	Lower
Exploration	Local	Broad
Exploitation	Strong	Weak

Practical guidance

A common starting point is a tenure of $\sqrt{n}$ to n (problem size). Dynamic tenure — randomised within a range at each step — can outperform a fixed value by preventing predictable cycles. For TSP, $3n$ has been found empirically effective.

Example ctd.

- Initial assignment $\mathbf{x} = (0, 1, 1, 1, 0, 0, 0, 1)$
- After 4 additional iterations M :

3	0	1	5	0	4	2	0

- Most recent flip $M(4) = 5$
- Current solution: $\mathbf{x} = (1, 1, 0, 0, 0, 1, 1, 1)$ with $eval(\mathbf{x}) = 33$

Example ctd.

- Initial assignment $\mathbf{x} = (0, 1, 1, 1, 0, 0, 0, 1)$
- After 4 additional iterations M :

3	0	1	5	0	4	2	0

- Most recent flip $M(4) = 5$
- Current solution: $\mathbf{x} = (1, 1, 0, 0, 0, 1, 1, 1)$ with $eval(\mathbf{x}) = 33$

Neighborhood of $\mathbf{x}$

$\mathbf{x}_1 = (0, 1, 0, 0, 0, 1, 1, 1)$
$\mathbf{x}_2 = (1, 0, 0, 0, 0, 1, 1, 1)$
$\mathbf{x}_3 = (1, 1, 1, 0, 0, 1, 1, 1)$
$\mathbf{x}_4 = (1, 1, 0, 1, 0, 1, 1, 1)$

$\mathbf{x}_5 = (1, 1, 0, 0, 1, 1, 1, 1)$
$\mathbf{x}_6 = (1, 1, 0, 0, 0, 0, 1, 1)$
$\mathbf{x}_7 = (1, 1, 0, 0, 0, 1, 0, 1)$
$\mathbf{x}_8 = (1, 1, 0, 0, 0, 1, 1, 0)$

Example ctd.

- Initial assignment $\mathbf{x} = (0, 1, 1, 1, 0, 0, 0, 1)$
- After 4 additional iterations M :

3	0	1	5	0	4	2	0

- Most recent flip $M(4) = 5$
- Current solution: $\mathbf{x} = (1, 1, 0, 0, 0, 1, 1, 1)$ with $eval(\mathbf{x}) = 33$

Neighborhood of $\mathbf{x}$

$\mathbf{x}_1 = (0, 1, 0, 0, 0, 1, 1, 1)$
$\mathbf{x}_2 = (1, 0, 0, 0, 0, 1, 1, 1)$
$\mathbf{x}_3 = (1, 1, 1, 0, 0, 1, 1, 1)$
$\mathbf{x}_4 = (1, 1, 0, 1, 0, 1, 1, 1)$
$\mathbf{x}_5 = (1, 1, 0, 0, 1, 1, 1, 1)$
$\mathbf{x}_6 = (1, 1, 0, 0, 0, 0, 1, 1)$
$\mathbf{x}_7 = (1, 1, 0, 0, 0, 1, 0, 1)$
$\mathbf{x}_8 = (1, 1, 0, 0, 0, 1, 1, 0)$

TABU, best evaluation $eval(\mathbf{x}_5) = 32$, **decrease!**

Example ctd.

- Current solution: $\mathbf{x} = (1, 1, 0, 0, 0, 1, 1, 1)$ with $eval(\mathbf{x}) = 33$
- New solution: $\mathbf{x}_5 = (1, 1, 0, 0, 1, 1, 1, 1)$ with $eval(\mathbf{x}_5) = 32$

3	0	1	5	0	4	2	0

changes to:

2	0	0	4	5	3	1	0

Example ctd.

- Current solution: $\mathbf{x} = (1, 1, 0, 0, 0, 1, 1, 1)$ with $eval(\mathbf{x}) = 33$
- New solution: $\mathbf{x}_5 = (1, 1, 0, 0, 1, 1, 1, 1)$ with $eval(\mathbf{x}_5) = 32$

3	0	1	5	0	4	2	0

changes to:

2	0	0	4	5	3	1	0

Policy might be too restrictive

- What if tabu neighbor $\mathbf{x}_6$ provides excellent evaluation score?
- Make search more flexible: override tabu classification if solution is outstanding

$\Longrightarrow$ **aspiration criterion**

Aspiration Criterion

General Definition

A tabu move is **admissible** despite being tabu if it satisfies an **aspiration criterion** — a condition that overrides the tabu status because the move is considered **outstanding**.

Most common criterion:

Accept a tabu move producing neighbour y if y is strictly better than the best solution found so far:

$$eval(y) > eval(best)$$

This is the criterion used in the SAT and TSP examples in this lecture.

Intuition: the tabu restriction is meant to prevent **revisiting** already-explored regions. If a tabu move leads somewhere **strictly better** than anything seen so far, we cannot have visited that solution before — so the restriction is unnecessary and should be overridden.

Aspiration Criterion ctd.

Further aspiration criteria:

- **Aspiration by default**: if **all** moves are tabu, select the least-recently-tabu one (the oldest tabu move) rather than getting stuck entirely.
- **Aspiration by search direction**: accept a tabu move if the direction it represents has been consistently improving in recent iterations.
- **Aspiration by influence**: accept if the move produces a solution that is structurally very different from the current one (large distance), promoting diversification.

Important

Without aspiration by default, tabu search can **get stuck** when all neighbours are tabu and no admissible move exists. This criterion guarantees the search can always continue.

Long-term Memory

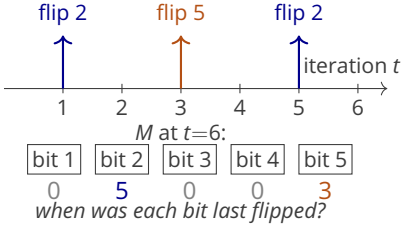
Question

1. What is stored in long-term memory (think of SAT as an example)?
2. How can we escape local optima with help of a long-term memory?



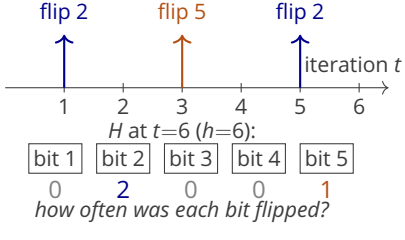
Recency-based vs. Frequency-based Memory

Recency-based memory *M*



- Tracks the **most recent** flip iteration per bit.
- Determines whether a flip is **tabu** ($t - M(i) \leq$ tenure).
- Short memory — forgets after the tenure window.

Frequency-based memory *H*



- Tracks the **total count** of flips per bit within the last h iterations.
- Used to **penalise** over-used moves and promote under-used ones.
- Long memory — operates over a larger horizon.

Frequency-based Memory

- Operates over a longer horizon
- SAT: vector H serves as long-term memory.
  - Initialized to 0, at any stage of the search

$$H(i) = j$$

    interpreted as: during last h (horizon) iterations, the i-th bit was flipped j times
  - Usually horizon is large
  - After 100 iterations with $h = 50$, long-term memory H might have the following values

5	7	11	3	9	8	1	6

  - Shows distribution of moves throughout the last 50 iterations

Diversity of Search

Frequency-based memory provides information about which flips have been under-represented or not represented.

$\Longrightarrow$ we can diversify the search by exploring these possibilities

Use of Long-term Memory

Special Circumstances

- Situations where all non-tabu moves lead to worse solution
- To make a meaningful decision about which direction to explore next
- Typically: most frequent moves are less attractive
- Value of evaluation score is decreased by some penalty measure that depends on frequency, final score implies the winner

Example SAT

- Assume value of current solution is $eval(\mathbf{x}) = 35$
- Non-tabu flips $2, 3$ and 7 have values $30, 33, 31$
- None of tabu moves provides value greater than 37 (highest value so far)
  $\Longrightarrow$ we can't apply aspiration criterion

Example SAT

- Assume value of current solution is $eval(\mathbf{x}) = 35$
- Non-tabu flips $2, 3$ and 7 have values $30, 33, 31$
- None of tabu moves provides value greater than 37 (highest value so far)
  $\Longrightarrow$ we can't apply aspiration criterion
- Frequency based-memory and evaluation function for new solution $\mathbf{x}'$ is

$$eval(\mathbf{x}') - penalty(\mathbf{x}')$$

- $penalty(\mathbf{x}') = 0.7 \times H(i)$, where 0.7 coefficient, $H(i)$ value from long-term memory H :

7	for solution created by flipping 2nd bit
11	for solution created by flipping 3rd bit
1	for solution created by flipping 7th bit

Example SAT

- Assume value of current solution is $eval(\mathbf{x}) = 35$
- Non-tabu flips 2, 3 and 7 have values 30, 33, 31
- Frequency based-memory and evaluation function for new solution $\mathbf{x}'$ is

$$eval(\mathbf{x}') - penalty(\mathbf{x}')$$

- $penalty(\mathbf{x}') = 0.7 \times H(i)$, where 0.7 coefficient, $H(i)$ value from long-term memory H :

7	for solution created by flipping 2nd bit
11	for solution created by flipping 3rd bit
1	for solution created by flipping 7th bit

- New scores are:

$30 - 0.7 \times 7 = 25.1$	2nd bit
$33 - 0.7 \times 11 = 25.3$	3rd bit
$31 - 0.7 \times 1 = 30.3$	7th bit

Diversify Search

Diversify Search

Including frequency values in a penalty measure for evaluating solutions.

Further Options to Diversify Search

We might add additional rules:

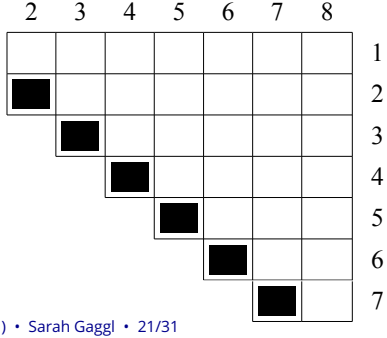
- Aspiration by default: select the oldest of all considered
- Aspiration by search direction: memorize whether or not the performed moves generated any improvement
- Aspiration by influence: measures the degree of change of the new solution
  a) in terms of the distance between old and new solution
  b) change in solution's feasibility, if we deal with a constraint problem
  – Intuition: particular move has a larger influence if a larger step was made from old to new solution

Tabu Search and the TSP

- Move: swap two cities in a particular solution
- Current solution: $(2, 4, 7, 5, 1, 8, 3, 6)$
- 28 neighbors $\binom{8}{2} = \frac{7 \cdot 8}{2} = 28$
- Recency-based memory: swap of cities i and j in i-th row and j-th column (for $i < j$)
- Maintain number of remaining iterations for which swap stays on tabu list
- Frequency-based memory: same structure; indicate totals of all swaps within horizon $h = 50$

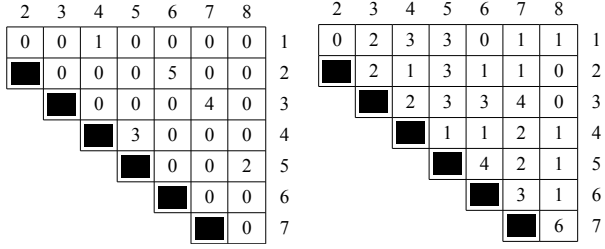
Tabu Search and the TSP

- Move: swap two cities in a particular solution
- Current solution: $(2, 4, 7, 5, 1, 8, 3, 6)$
- 28 neighbors $\binom{8}{2} = \frac{7 \cdot 8}{2} = 28$
- Recency-based memory: swap of cities i and j in i-th row and j-th column (for $i < j$)
- Maintain number of remaining iterations for which swap stays on tabu list
- Frequency-based memory: same structure; indicate totals of all swaps within horizon $h = 50$



Tabu Search and the TSP ctd.

- Assume both memories initialized to zero and 500 iterations have been completed
- Current solution: $(7, 3, 5, 6, 1, 2, 4, 8)$ with length: 173, best solution so far 171



left: recency-based memory; right: frequency-based memory

Particular Implementation of Tabu Search for TSP

Algorithm Tabu Search for TSP [Knox, J. (1994)]

```
tries ← 0
repeat
    generate a tour
    count ← 0
    repeat
        identify a set 𝔗 of 2-interchange moves
        select the best admissible move from 𝔗
        make appropriate 2-interchange
        update taub list and other variables
        if the new tour is the best-so-far for a given 'tries' then
            update local best tour information
        else
            count ← count + 1
        end if
    until count = ITER
    tries ← tries + 1
    if the current tour is the best-so-far (for all 'tries') then
        update global best tour information
    end if
until tries = MAX-TRIES
```

Particular Implementation of Tabu Search for TSP ctd.

- A tour is tabu if both added edges of interchange are on tabu list
- Tabu list update: placing added edges on list (deleted edges are ignored)
- Tabu list is of fixed size
- Whenever it is full, the oldest element in list is replaced by new deleted edge
- Initially, list is empty and all elements of aspiration list are set to large values
- Note: Algorithm examines all neighbors, i.e. all 2-*interchange* tours

Particular Implementation of Tabu Search for TSP ctd.

- A tour is tabu if both added edges of interchange are on tabu list
- Tabu list update: placing added edges on list (deleted edges are ignored)
- Tabu list is of fixed size
- Whenever it is full, the oldest element in list is replaced by new deleted edge
- Initially, list is empty and all elements of aspiration list are set to large values
- Note: Algorithm examines all neighbors, i.e. all 2-*interchange* tours
- Best results were achieved when
  - Length of tabu list was 3*n* (*n* number of cities of problem)
  - Candidate tour could override tabu status if both edges passed aspiration test. Compared length of tour with aspiration values for both added edges. If length of tour was better than both aspiration values, the test was passed.
  - Values present on aspiration list were tour costs prior to the interchange
  - MAX-TRIES and ITER (# of interchanges) depend on size of the problem. For 100 cities or less, MAX-TRIES was 4, and ITER was set to $0.0003 \cdot n^4$.

Simulated Annealing vs. Tabu Search

	Simulated Annealing	Tabu Search
Escape mechanism	Accepts worse moves probabilistically; probability decreases as T cools.	Accepts worse moves deterministically when no improving non-tabu move exists.
Determinism	Stochastic — different runs from the same start may differ.	Deterministic — same start always gives the same trajectory.
Memory	None — decisions depend only on current solution and T.	Explicit tabu list; optionally also frequency-based long-term memory.
Uphill moves	At any time, controlled by T.	Only when stuck: best admissible move may be worse than current solution.
Parameters	T_{max}, T_{min}, cooling schedule, inner loop length.	Tenure, neighbourhood, aspiration criterion, penalty coefficient.

Common ground

Both SA and tabu search work on **complete solutions**, can be halted at any iteration for a feasible answer, and have many parameters to tune. Neither guarantees finding the global optimum.

Computational Costs — SA vs. Tabu Search

	Simulated Annealing	Tabu Search
Neighbourhood scan	One random neighbour per step	All $\lvert N(x)\rvert$ neighbours per step
Cost per iteration	$\mathcal{O}(1)$ evaluation calls	$\mathcal{O}(\lvert N(x)\rvert)$ evaluation calls
Memory	$\mathcal{O}(1)$ — current solution and T only	$\mathcal{O}(n)$ vector-based (e.g. SAT); $\mathcal{O}(n^2)$ matrix-based (e.g. TSP)
Total iterations	Many — compensates for single-sample noise	Fewer — full scan gives best move each step
Parallelism	Easy — independent restarts share no state	Harder — tabu list is shared state

n = problem size; $\lvert N(x)\rvert$ = neighbourhood size.

Computational Costs ctd.

Neighbourhood size $|N(x)|$ is the key cost driver:

- For **SAT** with n variables: $|N(x)| = n$ — one flip per variable. Both SA and tabu search are cheap per iteration; tabu search's full scan costs $\mathcal{O}(n)$ evaluations, which is affordable even for large n.
- For **TSP** with n cities: $|N(x)| = \binom{n}{2} = \mathcal{O}(n^2)$ under 2-opt swaps. Tabu search scans all of them each step; SA samples just one. For $n = 1000$ this means tabu search performs $\sim$500 000 evaluations per iteration.
- For **large** n: scanning the full neighbourhood becomes the bottleneck. SA's $\mathcal{O}(1)$ per-iteration cost scales far better. **Neighbourhood pruning** (restricting the scan to a promising subset) is a common practical remedy for tabu search.

Rule of thumb

If $|N(x)|$ is small or moderate, tabu search's full scan pays off in solution quality per iteration. If $|N(x)|$ grows as $\mathcal{O}(n^2)$ or more, SA's cheap single-sample iterations are preferable — unless the neighbourhood can be effectively pruned.

Practical Guidelines — When to Use Which Algorithm

Prefer Tabu Search when…

- The problem has a **well-defined, small-to-moderate neighbourhood** that can be scanned fully each step (e.g. SAT, TSP with moderate n).
- **Reproducibility** is important: a deterministic algorithm is easier to debug, certify, and compare.
- You want to exploit **problem structure** through memory: frequency-based diversification steers the search away from over-explored regions.
- The problem has **combinatorial structure** with a natural move set (scheduling, graph colouring, routing).

Prefer Simulated Annealing when…

- The **neighbourhood is very large** ($\mathcal{O}(n^2)$ or more) and a full scan each step is too expensive.
- The search space is **continuous or high-dimensional**, where a tabu list is hard to define meaningfully.
- **Multiple independent runs** with different outcomes are desirable (solution diversity, statistical comparison).
- **Simplicity of implementation** is a priority: SA needs only an evaluation function and a perturbation operator — no memory bookkeeping.

Practical Guidelines ctd. — Hybrid Approaches

In practice the boundary between SA and tabu search is not sharp. Hybrid approaches that borrow ideas from both often outperform either algorithm used in isolation.

SA with memory elements

- **Restart from best**: after cooling, restart from the best solution found so far rather than a fresh random point, retaining the most useful information from the run.
- **Frequency penalty**: penalise frequently visited regions during acceptance, steering SA away from areas it has already explored thoroughly.

Tabu search with stochastic elements

- **Randomised tenure**: vary the tabu tenure randomly within a range $[\tau_{\min}, \tau_{\max}]$ at each step, preventing the search from falling into predictable cycles.
- **Random restarts**: periodically restart from a new random solution, combining tabu search's structured exploitation with the diversity of multiple starting points.

Key takeaway

Neither SA nor tabu search is universally superior. The best choice depends on the problem structure, neighbourhood size, and whether reproducibility or diversity of solutions is more important. When in doubt, **try both** and compare empirically.

Summary

- Simulated annealing and tabu search are both design to escape local optima
- Tabu search makes uphill moves only when it is stuck in local optima
- Simulated annealing can make uphill moves at any time
- Simulated annealing is stochastic, tabu search is deterministic
- Compared to classic algorithms, both work on complete solutions. One can halt them at any iteration and obtain a possible solution
- Both have many parameters to worry about

Outlook

Next lecture: Evolutionary Algorithms

References

- Zbigniew Michalewicz and David B. Fogel.
  How to Solve It: Modern Heuristics, volume 2. Springer, 2004.
- Knox, J.
  Tabu Search Performance on the Symmetric Traveling Salesman Problem, Computer Operations Research, Vol.21, No.8, pp.867–876, 1994.